

Seven Segment Display Interfacing with ESP32-WROOM Board

Implementing a Simplified Boolean Expression via 7447 Decoder IC

Aim

To interface a seven segment display with the ESP32-WROOM development board through a 7447 BCD-to-seven-segment decoder IC, by implementing the simplified Boolean expression $f = P \text{ xor } Q \text{ xor } R$ on the ESP32's GPIO output, and to verify on hardware that the display shows the correct digit - '0' for logic LOW and '1' for logic HIGH - in agreement with the derived truth table.

GATE Question

The Boolean expression for the output f of a 4:1 multiplexer with inputs $I_0 = R$, $I_1 = R'$, $I_2 = R'$, $I_3 = R$ and select lines $S_1 = P$, $S_0 = Q$ is one of the following:

Option	Expression
A	$(P \text{ xor } Q \text{ xor } R)'$
B	$P \text{ xor } Q \text{ xor } R$
C	$P + Q + R$
D	$(P + Q + R)'$

Question Analysis

1. Select-Line to Output Mapping

$S_1 (P)$	$S_0 (Q)$	Line Selected	Output f
0	0	I_0	R
0	1	I_1	R'
1	0	I_2	R'
1	1	I_3	R

2. Sum of Products Form

$$f = P'Q'R + P'QR' + PQ'R' + PQR$$

3. Observation and Final Expression

f is 1 only where P , Q , R contain an odd number of 1's (001, 010, 100, 111) - the defining property of the three-input XOR operation.

Therefore, $f = P \text{ xor } Q \text{ xor } R$, which matches **Option (B)**.

Correct Answer: (B) $f = P \text{ xor } Q \text{ xor } R$

Components Used

S.No.	Component
1	ESP32-WROOM Development Board
2	USB Cable
3	Breadboard
4	Seven Segment Display
5	Jumper Wires

Purpose of Each Component

Component	Function
ESP32-WROOM Development Board	Generates the BCD output corresponding to the result of the Boolean expression and drives it onto the GPIO pins.
USB Cable	Connects the ESP32 board to the computer for flashing the firmware and powering the board.
Breadboard	Provides a platform to build and hold the complete circuit.
Seven Segment Display	Displays the decoded digit ('0' or '1') representing the output of the Boolean expression.
Jumper Wires	Connects the ESP32 board, decoder circuitry, and display together.

Hardware Connections

S.No.	Connection Description
1	ESP32 GPIO16 -> 7447 Input A
2	ESP32 GPIO17 -> 7447 Input B
3	ESP32 GPIO18 -> 7447 Input C
4	ESP32 GPIO19 -> 7447 Input D
5	ESP32 GND -> 7447 GND
6	ESP32 5V (VIN) -> 7447 VCC
7	7447 Outputs (a-g) -> 7-Segment Display Segments (a-g)
8	Common Anode of 7-Segment -> +5V Supply
9	Segment Lines -> Through 220 Ohm Resistors
10	ESP32 USB Port -> Ubuntu PC (Code Upload)

Execution Procedure

1. A PlatformIO project was set up for the ESP32-WROOM board (board type esp32dev).
2. The source code implementing the Boolean expression $f = P \text{ xor } Q \text{ xor } R$, and outputting the result in BCD form on GPIO16-GPIO19, was placed inside the src folder of the project.
3. The project was built to generate the firmware, bootloader, and partition binary files (bootloader.bin, partitions.bin, firmware.bin).
4. The generated binaries were copied to the desired location on the Ubuntu PC.
5. The ESP32 board was connected to the Ubuntu PC through a USB cable, and the system was checked to confirm the board was detected as a serial device (/dev/ttyUSB0).
6. The chip identity was verified using the esptool.py chip_id command.
7. The generated binary files were flashed onto the ESP32 at their respective memory offsets using esptool.py write_flash.
8. Once flashing completed, the EN (reset) button on the board was pressed to run the new firmware.
9. The seven segment display, driven by the 7447 decoder from the ESP32's BCD output, was observed to confirm that the displayed digit matched the expected logic output of the Boolean expression.

Truth Table

P	Q	R	f	7-Seg Display
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	0
1	0	0	1	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Results and Observations

The simplified Boolean expression $f = P \text{ xor } Q \text{ xor } R$ was successfully implemented on the ESP32-WROOM development board. Its output was converted into BCD form and applied to the 7447 BCD-to-seven-segment decoder IC, which in turn drove the common anode seven segment display. After flashing the generated binaries using esptool.py, the display showed digit '0' whenever the output f was logic LOW and digit '1' whenever f was logic HIGH, exactly matching the truth table derived from the GATE question for all input combinations. This confirmed that the 4:1 multiplexer expression simplifying to the three-input XOR function was correctly realized in hardware using the ESP32 board together with the 7447 decoder and seven segment display.